

Q1. What is React?

React is a JavaScript library for building user interfaces using reusable components.

Features of React

- Component-based architecture
- Virtual DOM
- Reusable UI
- One-way data binding
- JSX support
- Fast rendering

Q2. What is JSX?

JSX stands for JavaScript XML. It allows writing HTML-like syntax inside JavaScript.

Eg:- `const element = <h1>Hello</h1>;`

Q3. Difference between React and JavaScript

React is a library built on JavaScript for UI development.

Q4. What is Virtual DOM?

Virtual DOM is a lightweight copy of the real DOM. React compares changes and updates only necessary parts.

Q5. Why React is fast?

Because React uses:

- Virtual DOM
- Component reusability

Q6. What are components?

Reusable independent UI blocks.

Types of components

1. Functional Components
2. Class Components

Functional Component Example -

```
function App() {  
  return <h1>Hello</h1>;  
}
```

Class Component Example -

```
class App extends React.Component {  
  render() {  
    return <h1>Hello</h1>;  
  }  
}
```

Q7. Difference between Functional and Class Components

Functional	Class
Simpler	More boilerplate
Hooks supported	Lifecycle methods
Preferred now	Older approach

Q8. What are Props?

Props are read-only data passed from parent to child.

Eg- function User(props){
 return <h1>{props.name}</h1>
}

Q9. What is State?

State stores dynamic component data.

Eg- const [count, setCount] = useState(0);

Q10. Difference between Props and State

Props	State
Immutable	Mutable
Passed by parent	Managed inside component
Read-only	Can change

Q11. What are Hooks?

Hooks allow functional components to use state and lifecycle features.

Q12. Why `useState` does not update immediately?

Because React batches state updates and updates the UI on the **next render cycle** for performance optimization.

- When you call `setState`, React:
 1. Schedules the update
 2. Finishes the current function execution
 3. Re-renders the component with the new value
- React uses **asynchronous state updates** because:
 1. Multiple updates can be grouped together
 2. Prevents unnecessary re-renders
 3. Improves performance

This is called **batching**.

Why this fails?

```
setCount(count + 1);  
setCount(count + 1);
```

Because both calls use same old value of `count`.

Correct Solution -

Use functional update:

```
setCount(prev => prev + 1);  
setCount(prev => prev + 1);
```

Now result becomes: 2 Because React uses latest updated state each time.

Q13. What is useEffect ?

`useEffect` is a React Hook used to perform **side effects** in functional components.

Side effects are things like:

- API calls
- Fetching data
- Updating DOM
- Timers (`setInterval`)
- Event listeners
- Local storage operations

Syntax - `useEffect(() => {`
`}, [dependencies]);`

Q14. Why useEffect is needed?

Functional components do not have lifecycle methods like classes.

`useEffect` handles:

- Mounting
- Updating
- Unmounting

Infinite Loop Mistake

✗ Wrong:

```
useEffect(() => {  
  setCount(count + 1);  
});
```

This causes infinite re-renders.

Because:

- Effect runs
- State updates
- Component re-renders
- Effect runs again

Q15. useEffect Dependency Array

Dependency	Behavior
[]	Run once
[value]	Run on value change
none	Run every render

Q16. What is useRef?

`useRef` is a React Hook used to:

1. Access DOM elements directly
2. Store mutable values without re-rendering the component

Syntax - `const refName = useRef(initialValue);`

Eg - Accessing DOM Elements -

Example: Focus input automatically

```
import React, { useRef } from "react";

export default function App() {
  const inputRef = useRef();

  function focusInput() {
    inputRef.current.focus();
  }

  return (
    <div>
      <input ref={inputRef} type="text" />

      <button onClick={focusInput}>
        Focus Input
      </button>
    </div>
  );
}
```

Unlike `useState`, updating `useRef` does NOT re-render component.

- Example:
- `import React, { useRef, useState } from "react";`

```
export default function App() {
  const countRef = useRef(0);
  const [state, setState] = useState(0);

  function increaseRef() {
    countRef.current++;
    console.log(countRef.current);
  }

  function rerender() {
    setState(countRef.current);
  }

  return (
    <div>
      <h1>State: {state}</h1>

      <button onClick={increaseRef}>
        Update Ref
      </button>

      <button onClick={rerender}>
        Re-render
      </button>
    </div>
  );
}
```

- Updating:
 - `countRef.current++`
 - does NOT update UI.
 - Because `useRef` does not trigger re-render.

Q17 . Difference Between `useState` and `useRef`

Feature	<code>useState</code>	<code>useRef</code>
Causes Re-render	✓ Yes	✗ No
Stores Value	✓ Yes	✓ Yes
Used for DOM Access	✗ No	✓ Yes
Persistent Between Renders	✓ Yes	✓ Yes

Q18. Common useRef Use Cases

Focus Input

```
inputRef.current.focus();
```

means - "Go to the actual input element and place the cursor inside it."

Store Previous Value

```
const prevCount = useRef();
```

Timer / Interval IDs

```
const timerRef = useRef();
```

Avoid Re-render for Temporary Data

Example:

- Mouse position
- Scroll position
- Previous state

Q19. Why useRef does not re-render?

Because React does not track changes inside:

ref.current

It is just a mutable object:

```
{  
  current: value  
}
```

Changing it does not notify React.

Q20. useRef vs Normal Variable

✗ Normal variable resets on every render:

```
let count = 0;
```

✓ useRef persists value across renders:

```
const countRef = useRef(0);
```

Q21. What is useMemo?

`useMemo` is a React Hook used to **memoize expensive calculations** so they are not recalculated on every render.

It improves performance.

Syntax-

```
const memoizedValue = useMemo(() => {  
  return expensiveCalculation();  
}, [dependencies]);
```

Q22. Why useMemo?

Normally, whenever component re-renders:

- Entire function runs again
- Heavy calculations run again

This can slow app performance.

`useMemo` stores (caches) previous calculated value.

Example –

```
const result = useMemo(() => {  
  console.log("Calculating...");  
  
  for (let i = 0; i < 10000000000; i++) {}  
  
  return count * 2;  
  
}, [count]);
```

Simple Analogy -

Without `useMemo`:

- Solve same math problem every time

With `useMemo`:

- Save previous answer in notebook
- Reuse until values change

Q23. Difference Between useMemo and useEffect

useMemo	useEffect
1.Returns memoized value	Performs side effects
2.Used for optimization	Used for lifecycle/side effects
3.Runs during render	Runs after render

Q24. Does useMemo prevent re-render?

✗ No

It only prevents:

- expensive recalculation

Component still re-renders.

Q25. useMemo vs useCallback

Hook	Memoizes
useMemo	Value
useCallback	Function

When to Use useMemo

- ✓ Expensive calculations
- ✓ Filtering large lists
- ✓ Sorting huge arrays
- ✓ Performance optimization

When NOT to Use

- ✗ Small/simple calculations

Q26. Why useCallback?

`useCallback` is a React Hook used to **memoize a function** so that the function is not recreated on every render.

It is mainly used for **performance optimization**.

Syntax

```
const memoizedFunction = useCallback(() => {  
  }, [dependencies]);
```

Q27. Why useCallback?

In React:

Every time component re-renders:

```
function handleClick() {}
```

gets recreated.

Sometimes this causes unnecessary child component re-renders.

EXAMPLE –

```
const handleClick = useCallback(() => {  
  console.log("Clicked");  
}, []);
```

Q28. What useCallback Does?

It stores the SAME function reference between renders, until dependencies change.

Important Note -

Usually used with:

```
React.memo()
```

because child component skips re-render if props are same.

When to Use useCallback

- ✓ Passing function to child components
- ✓ Prevent unnecessary renders
- ✓ Stable function references
- ✓ Optimization

When NOT to Use

✗ Everywhere blindly

This is unnecessary:

```
const add = useCallback(() => {  
  return 2 + 2;  
}, []);
```

Because memoization also has cost.

Q29. Does `useCallback` stop component re-render?

✗ No

It only memoizes function reference.

Q30. What is `useContext`?

`useContext` is a React Hook used to **share data globally** between components without passing props manually at every level.

This solves:

Prop Drilling

Steps to Use Context API

1. Create Context
2. Provide Context
3. Consume Context

Step 1 — Create Context

```
import { createContext } from "react";  
  
export const UserContext = createContext();
```

Step 2 — Provide Context

```
import React from "react";  
import { UserContext } from "../UserContext";  
import Child from "../Child";
```

```
export default function App() {  
  
  const user = "Sudhanshu";  
  
  return (  
    <UserContext.Provider value={user}>  
      <Child />  
    </UserContext.Provider>  
  );  
}
```

Step 3 — Consume Context Using useContext

```
import React, { useContext } from "react";  
import { UserContext } from "../UserContext";  
  
export default function Child() {  
  
  const user = useContext(UserContext);  
  
  return <h1>{user}</h1>;  
}
```

Output

Sudhanshu

without passing props manually.

Common Real-World Uses

- ✓ Authentication
- ✓ Theme (dark/light mode)
- ✓ Language settings
- ✓ Cart data
- ✓ User profile
- ✓ Global state

Q30. What is Prop Drilling?

Passing props through multiple components unnecessarily.

Example:

App → Parent → Child → GrandChild

If `GrandChild` needs data from `App`,
you pass props through every component. This becomes messy.

Q31. Difference Between Props and Context

Props	Context
Passed manually	Global sharing
Good for small hierarchy	Good for deeply nested components
Causes prop drilling	Avoids prop drilling

Important Note-

When context value changes:

```
value={newValue}
```

all consuming components re-render.

Q32. What is useReducer?

`useReducer` is a React Hook used for **complex state management**.

It is an alternative to:

`useState`

especially when:

- state logic is complex
- multiple state updates exist
- next state depends on previous state

Syntax-

```
const [state, dispatch] = useReducer(reducer, initialState);
```

Important Terms-

Term	Meaning
reducer	Function that updates state
state	Current state
dispatch	Sends action
action	Object describing what to do

Basic Flow-

```
dispatch(action)
  ↓
reducer(state, action)
  ↓
new state returned
  ↓
UI updates
```

Basic Example — Counter-

```
import React, { useReducer } from "react";

const initialState = 0;

function reducer(state, action) {

  switch(action.type) {

    case "increment":
      return state + 1;

    case "decrement":
      return state - 1;

    default:
      return state;
  }
}

export default function App() {

  const [count, dispatch] = useReducer(
    reducer,
    initialState
  );

  return (
    <div>
      <h1>{count}</h1>

      <button
        onClick={() =>
          dispatch({ type: "increment" })
        }
      >
      +
      </button>

      <button
        onClick={() =>
          dispatch({ type: "decrement" })
        }
      >
      - </button> </div> ); }
```

Q33. Why useReducer?

Better for:

- ✓ Complex state logic
- ✓ Multiple related states
- ✓ Large forms
- ✓ Authentication state
- ✓ Shopping cart
- ✓ Predictable updates

Q34. useState vs useReducer

useState	useReducer
Simple state	Complex state
Easy syntax	Structured logic
Small apps	Large apps
Multiple setters	Single dispatch

Q35. Example of useReducer

Real Example with Object State

```
const initialState = {  
  count: 0,  
  theme: "dark"  
};
```

Reducer:

```
function reducer(state, action) {  
  
  switch(action.type) {  
    case "increment":  
      return {  
        ...state,  
        count: state.count + 1  
      };  
    case "toggleTheme":  
      return {  
        ...state,  
        theme:  
          state.theme === "dark"  
            ? "light"  
            : "dark";  
      };  
  
    default:  
      return state;  
  }  
}
```

Action Payload-

Actions can carry data.

```
dispatch({
  type: "add",
  payload: 5
});
```

Reducer:

```
case "add":
  return state + action.payload;
```

Q36. Why reducer must be pure?

Reducer should:

- not modify original state
- not call APIs
- not use async code

Because React expects predictable state updates.

Common Pattern

```
const ACTIONS = {
  INCREMENT: "increment",
  DECREMENT: "decrement"
};
```

Avoids typo mistakes.

useReducer + useContext

Very common combination for:

- global state management
- Redux-like architecture

Example:

- auth state
- cart system
- app settings

